



NATURAL LANGUAGE PROCESSING

FINAL PROJECT REPORT

When does multi-agent LLM coordination create value?
An empirical study under three controls

Dương Trọng Nguyên
24022420

Trương Đình Đức
24021419

Trần Tùng Dương
24022309

Lê Hoàng Quân
24022433

Course code: INT3406

Hanoi, August 2026

ACKNOWLEDGEMENT

We would like to express our sincere and deepest thanks to our lecturer, **Trần Hồng Việt**, who taught the Natural Language Processing course at VNU University of Engineering and Technology.

Over the past semester, his dedicated lectures gave us a firm grounding in the subject and in current approaches to large language models. More than that, he trained us in rigorous scientific thinking, in a critical attitude towards our own results, and in a demanding standard of fairness and discipline when running empirical evaluations.

This report is the product of serious and sustained work by the whole group. To get a clear answer to our research question, we spent a great deal of time discussing the design together, setting up controlled comparison conditions, and going through failure cases one by one. The experimental work ran into more than a few obstacles, both methodological and in the compute available to us, but persistence and a willingness to keep learning let the group carry the project through as fully as it could.

Despite our best efforts, this report certainly still has limitations and shortcomings. We would be very glad to receive further comments and guidance from him, so that we can understand the problem more deeply and continue to improve as we go on with our studies.

We wish him continued good health, happiness, and much success in his teaching and in his research.

Hanoi, August 2026

The student group

Dương Trọng Nguyên	— 24022420
Trương Đình Đức	— 24021419
Trần Tùng Dương	— 24022309
Lê Hoàng Quân	— 24022433

Abstract

Multi-agent LLM systems have drawn growing interest because they let a task be divided across roles such as planning, solving, verifying, and aggregating, on the expectation that the resulting pipeline does better than a single model working alone. The value that coordination itself adds is hard to assess, however. Reported comparisons differ in compute budget, in which model serves as the control, and in how wide an effect they claim. In this report we build one experimental framework over reasoning and coding benchmarks and use it to study three questions: how much each role contributes, what happens when one model’s solution is passed to another, and whether a model can be trained to carry out its assigned role. We find that most of the reported benefit shrinks or disappears once the comparison is made on equal terms. In particular, an incorrect solution from a weak model can sharply reduce the accuracy of a strong model that reads it, while an objective verification signal is more reliable than asking a model to judge its own output. Taken together, the measurements suggest that multi-agent coordination does not produce a benefit by default. Whether it helps depends on the capability of the models involved, on how information is routed between them, and on the mechanism used to select and verify candidate solutions.

1 Introduction

We asked a 1.5B model to write an implementation plan and told it explicitly not to write code. It wrote code anyway in 53.7% of HumanEval problems; told to write it, in 100%. Adding that planner to the pipeline made the pipeline worse: 0.5375 accuracy with no planner against 0.4312 with one, in five folds out of five. The instruction that defines the role is the instruction the model does not follow, and here following it or not was not the difference between a good plan and a bad one — it was the difference between having the component and not.

That is a small observation with a large consequence for how multi-agent systems are measured. If the component labelled *planner* is in fact solving, then a measurement of what the planner contributes is not a measurement of planning. It is a measurement of a second solving attempt wearing a different name. The same doubt applies to every other role: a verifier that re-derives the answer instead of checking it, an aggregator that generates rather than selects. So before asking whether a Planner–Solver–Verifier–Aggregator pipeline beats a single model, there is a prior question, and it is the question this report asks: when one model is split into four named roles, do the roles exist?

The usual intuition says that they do, and that dividing the work helps — split the task, add cross-checking, catch more errors. Most recent published work in the area supports that intuition. Our measurements at a single model size do not. What looks like a gain from coordination is, for the most part, the gain from sampling the same model more than once (§5.2).

Reading the evidence for or against that intuition requires three controls on how the benefit is measured. Published comparisons routinely omit at least one.

First, compute budget. With the same 1.5B model, a four-role pipeline emits 2.9 times (on GSM8K) to 6.6 times (on MATH) as many tokens as a single generation. When the roles use models of different sizes, even tokens are not priced alike – a token generated by a 7B model costs roughly 4.7–4.9 times the arithmetic (FLOPs) of a token from a 1.5B model, because inference cost scales with parameter count [12] (§5.8). Comparing an expensive system against a single model without normalizing both to the same compute budget (FLOPs) yields an unfair verdict on efficiency.

Second, choice of baseline. Most generate-and-repair work measures improvement against the model being repaired (the weak model). In practice, though, the decision a deployer faces is whether to run the whole multi-agent system or just a single strong model. This report shows that changing the baseline can reverse the sign of the measured effect entirely (§5.8, §6).

Third, denominator. On MATH with the 1.5B model, 57% of problems lie outside anything a candidate-selection mechanism could change. Of these, 32% are too hard, so every candidate answer is wrong, and

25% are too easy, so every candidate answer is right. In neither case can an aggregation method improve anything. Averaging over the full dataset dilutes the real improvement signal and risks wrongly rejecting interventions that do work (§5.7). Omitting this control *underestimates* the true effect; omitting the other two *overestimates* it — the three biases do not point the same way, so all three must be applied together and none substitutes for another.

Contributions. This report unifies three research lines into a single measurement frame: decomposing contributions by role, analysing the cost and benefit of routing, and analysing information flow. There are four main contributions:

1. **A credit decomposition that holds the solver fixed.** Treating the four roles as players and taking the solver as given, we measure what the remaining three add. Planner, verifier and aggregator together add 4.9 points on GSM8K and nothing measurable on MATH (§5.3).
2. **Direct evidence that the role boundaries are not enforceable by instruction**, on maths and on code. A planner told not to write code writes it in 53.7% of HumanEval problems, and in 100% when told to; on maths the plan’s final number already matches the gold answer in 33.3% of GSM8K problems; a planner told not to write code writes it in 53.7% of HumanEval problems, and in 100% when told to. On HumanEval the planner is net harmful: 0.5375 with no planner against 0.4312 with one, with the no-planner condition at or above the planner condition in all five folds (§5.4).
3. **Three controls.** We give empirical evidence that when all three controls (compute budget, choice of baseline, and denominator) are applied together, most of the positive effects of multi-agent coordination shrink or reverse (§5).
4. **A pre-registered experiment on why a stronger checker is not the remedy.** The obvious repair for a weak role is to bring in a stronger model to check the work. That move carries its own cost: on the stratum where the weak model’s solution is wrong, showing that solution to the strong model drops the strong model’s accuracy by 27.2 points, from 46.4% to 19.2% (Figure 1, §5.9).

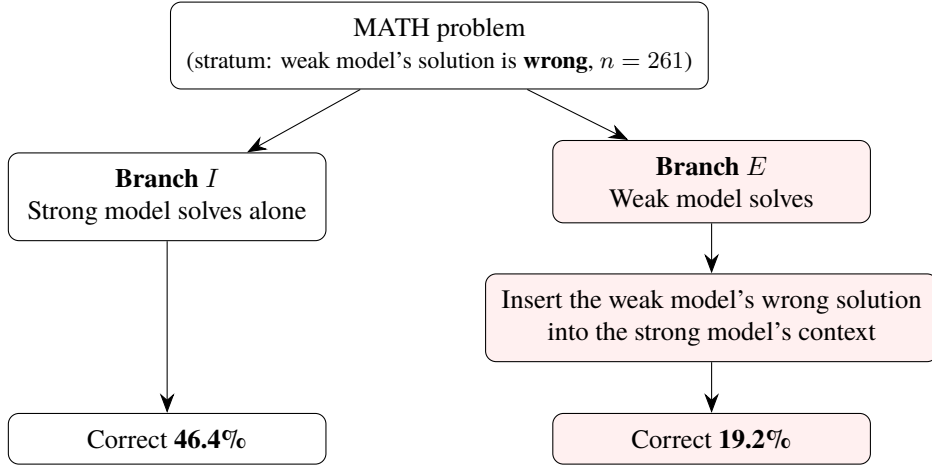


Figure 1: *Artifact-exposure experiment* — showing the strong model the weak model’s solution (pre-registered design, MATH-500): the two branches differ only in whether the context carries the weak model’s wrong solution. The gap of -27.2 points ($p \approx 0$) is what a stronger checker costs when it is placed downstream of a weaker one (§5.9).

The results were collected under a reliability protocol that fixes the criteria in advance and discards runs that fail the validity conditions; the protocol is described in §4. Of 32 runs in total, 16 (50%) were discarded before the results were read. The report uses some notation of its own; Table 1 is the quick reference — every symbol is defined in full in §3–4 before it is used.

Notation quick reference (details: §3–4)

P, S, V, A	The four roles: plan – solve – verify/repair – select answer
$H / \kappa / D$	Three questions: headroom / exploitability / damage
$W / I / E$	Weak model / strong model alone / strong model that has seen W
artifact	The weak model’s solution (noise injected into the context)
G, L, R	Opportunity (G), loss (L), rescue (R)
Δ_{ceil}	Oracle ceiling $\Delta_{\text{ceil}} = G - L + R$
Δ_{real}	Effect as deployed: $\text{acc}(E) - \text{acc}(I)$
gap; g^*	Accuracy difference (strong – weak); sign-flip point of Δ_{ceil} (≈ 0.09)
generation	One full solution sampled from a model; the k generations are independent and equally weighted
<code>maj@k</code> ; <code>oracle@k</code>	Majority voting; oracle ceiling (k generations)
<code>exec3 / llm3</code>	Candidate selection by: running tests / LLM self-review
<code>V_gain, A_gain</code>	Change in accuracy when role V or A is added
points; CI	Percentage points (+14.0 points = +0.140); 95% confidence interval
noise floor	Spread when the same configuration is re-measured (up to 7 points per 100 problems)
fold; Level A / B	A disjoint block of 60–100 problems, five in total; 5-fold validation (t -test) / single illustrative measurement

Table 1: Quick reference for the notation and terminology used in this report.

2 Related work

Generate then select. Chain-of-thought prompting [22] set the stage for self-consistency [21], in which the system samples several independent solutions and settles the answer by majority voting — exactly the `maj@k` baseline of this report (defined in §4). Verifier-based selection then branched in two main directions: scoring the reasoning process step by step [13] and allocating extra compute at inference time [20]. The defining feature of this family is that the aggregation stage only picks among solutions that have already been generated; it does not write new ones. By the decomposition identity in §3.3, this family therefore carries no risk of destroying solutions that were already correct.

Generate then repair. Frameworks such as Self-Refine [14], Reflexion [19] and CRITIC [7] have a model read its own output, or another model’s, and then revise it. Huang et al. [11] showed that LLMs cannot yet correct their own reasoning errors without an external, objective verification signal, which is consistent with our results in §5.10. What matters most for this report is the *baseline*: in our review, most of this work reports improvement over the model *being repaired* or over the first draft, that is, over the weak model rather than over simply running the repairing model alone. This report shows that the two baselines give opposite signs on the same system (§5.8, §6).

Multi-agent systems and debate. Multi-agent debate [6] and LLM-as-a-judge [24] are common coordination mechanisms. A systematic review [15] that evaluated five debate frameworks across nine benchmarks concluded that these frameworks *do not consistently beat* self-consistency or even plain CoT, despite spending more compute; on Llama-3.1-8B on GSM8K, debate lost about 16 points relative to CoT (80.13 $\rightarrow$ 63.87) — consistent with the pattern of degradation on small models that we observed independently (§5.4). The two systems closest to ours in architecture and method are MAS_RPSV (a sequential Reader–Planner–Solver–Verifier configuration, surveyed by MASPRM [16] on GSM8K/MATH) and SHARP [17].

Where this report sits. The work above asks whether a multi-agent pipeline improves accuracy, and answers it by comparing end-to-end scores. This report asks the prior question: whether the roles inside such a pipeline are doing the jobs they are named for. We do not propose a new architecture. We propose

an empirical measurement frame — three controls, a decomposition identity, and a three-factor analysis — and use it to separate the value a role adds as a role from the value it adds as one more generation.

3 Measurement framework

This section sets out how we measure whether splitting one model into roles is worth anything. It has four parts. The first states the measurement problem and the three controls the report applies to any measured benefit: compute budget, choice of baseline, and denominator. The second defines how the outcome of a pipeline is attributed to an individual role; that is the machinery behind most of §5. The third is a smaller, self-contained sub-framework used by one experiment and by nothing else. The fourth classifies protocols by whether they can write a new solution or only choose among existing ones, a distinction that later sections refer back to.

3.1 Three controls, and why they have to be applied together

Comparing a role-structured pipeline against a single model is not a comparison of two accuracies. The two configurations differ in how much compute they spend; the reference the pipeline is scored against is a choice, not a given; and both numbers are averages over a problem set much of which no mechanism could move. Each of these is a separate control, and each moves the measured benefit on its own.

First, compute budget. A four-role pipeline calls the model several times per problem and emits several times as many tokens as a single generation (§5.1). When the roles are filled by models of different sizes, even tokens are not priced alike: inference cost scales with parameter count, so a token generated by a 7B model costs roughly 4.7–4.9 times the arithmetic of a token from a 1.5B model [12]. *Operationally*: where the question is about a mechanism, we hold the number of generations fixed and vary only the mechanism that consumes them (§5.2); where the question is about a configuration, we report cost on the FLOP scale (parameters $\times$ tokens) whenever the configurations being compared contain models of different sizes, and quote raw token counts only within one model size (§5.8). Comparing an expensive system against a cheap one at unequal budget overstates the benefit of the expensive one.

Second, choice of baseline. Most generate-and-repair work reports improvement over the model *being repaired*, which is the weaker of the two models involved. That is not the choice a deployer faces. The choice is between running the whole multi-agent system and running the strongest single model available, with the costs of the two put on a common scale. *Operationally*: every configuration that mixes model sizes is measured against both references where we have both — against the weak model, as the literature does, and against the strong model running alone — and both are reported (§5.8, §6). The two references can disagree in sign on the same system, and the one common in the literature is the more favourable of the two.

Third, denominator. An average over a whole benchmark includes problems on which no selection or aggregation method could have changed the outcome. *Operationally*: we stratify the problem set by how many of the k generations for that problem are correct. The $0/k$ stratum contains no correct candidate to select and the k/k stratum has nothing left to improve, so neither can move under any method that chooses among candidates; for majority voting the $1/k$ stratum is unreachable as well, since a single vote cannot carry a majority. We report the effect on the reachable strata alongside the full-set average (§5.7). The strata are read off the generations after the fact, so this is an interpretive tool, not a deployable baseline.

The three controls do not point the same way. Leaving out the cost control or the baseline control *inflates* the measured benefit; leaving out the denominator control *deflates* it, and can make a real intervention look like nothing. They therefore have to be applied together, and none of them substitutes for another: a result that survives one control has not been shown to survive the others. Underneath all three sits the noise

floor of the measurement itself (§4); a point estimate smaller than the fold-to-fold spread carries little information whichever controls were applied.

3.2 Assigning credit to a role, with the solver held fixed

The pipeline has four roles: P (planner — draft an outline), S (solver — produce a solution), V (verifier — check the solution, and allowed to repair it), A (aggregator — take several candidates and pick the final answer). A *coalition* is an on/off choice over these four roles, run as a pipeline in that fixed order with the absent roles skipped. We write coalitions as concatenations: $PSVA$ is all four roles, SA is the solver followed by the aggregator. The value of a coalition is its accuracy on the benchmark.

Credit is then split among the roles that produced that value. One convention governs the split: the **solver is held fixed**, treated as part of the setup rather than as a player, and the credit is divided among the other three. The reason is conceptual rather than statistical. Planner, verifier and aggregator are defined only relative to an existing solution — a verifier with nothing to check is not verifying, and an aggregator with no candidates to choose between is not aggregating. The quantity being divided is therefore the accuracy of the full pipeline minus the accuracy of the solver running alone, and the players are P , V and A .

We ran all $2^4 = 16$ on/off coalitions of the four roles and use the eight that contain the solver. Within that conditioned game, the contribution φ_i of role i is the mean change in accuracy when i is added to every sub-coalition it is absent from; it is the only allocation satisfying the efficiency, symmetry and additivity axioms [18], which is why the three values sum exactly to pipeline accuracy minus solver accuracy. The derived quantities V_gain and A_gain (§4) are the single-coalition version of the same comparison: accuracy with the role minus accuracy without it, holding everything else fixed.

One limitation is built into the construction and shapes the rest of the report. The decomposition assigns credit to the role *label*, not to the function actually performed. If the planner quietly solves the problem, the accuracy that follows is still booked under “planner”. The decomposition can therefore say how much a stage is worth, but not whether the stage does what it is named for. We pair it with a second class of measurement — trace statistics and prompt ablations that compare executed behaviour against assigned behaviour (§5.4).

3.3 A sub-framework for the artifact-exposure experiment

The notation below serves one experiment — §5.9, which asks what happens when the checking role is handed to a stronger model that is shown a weaker model’s solution. Nothing from §5.1 to §5.8 uses it.

Headroom, exploitability, damage. Whether such a mechanism creates value splits into three independent components: **headroom** (H) — does the pool contain a solution the strong model does not find on its own? **exploitability** (κ) — if it does, can a deployable signal, one that does not need the gold answer, pick that solution out (§5.10)? **damage** (D) — does the protocol break problems that were already solved, and by how much? Net value requires all three at once.

Symbols. Take a pair of models, one weak and one strong, say Qwen-1.5B and Qwen-7B. The three symbols below are deliberately different from the roles $P/S/V/A$ in §4, to keep the two sets from being confused:

- W — the **weak** model solving the problem on its own; the solution it produces is called the *artifact*.
- I — the strong model solving the problem **independently**, without seeing W ’s artifact.
- E — the same strong model, with the same solve prompt, but **exposed** to W ’s artifact in its input context.

I and E are the same model on the same task, differing only in whether the artifact is in context, so $\text{acc}(E) - \text{acc}(I)$ measures the effect of exposure directly. We write it Δ_{real} ; it needs no gold answers and so can be measured in a real deployment.

The decomposition identity. An ideal selector that knows the gold answer is correct whenever W or E is correct, so it reaches $\text{acc}(\text{CEIL}) = P(W \text{ correct} \vee E \text{ correct})$. Writing Δ_{ceil} for its maximum improvement over the strong model running alone,

$$\Delta_{\text{ceil}} = \text{acc}(\text{CEIL}) - \text{acc}(I) = G - L + R, \quad (1)$$

$$G = P(W \text{ correct} \wedge I \text{ wrong})$$

$$L = P(W \text{ wrong} \wedge I \text{ correct} \wedge E \text{ wrong})$$

$$R = P(W \text{ wrong} \wedge I \text{ wrong} \wedge E \text{ correct})$$

opportunity — a property of the *model pair*,

damage — caused by the *protocol*,

rescue — problems both models missed at first.

Identity (1) follows algebraically from expanding $P(W \vee E) - P(I)$. It splits the maximum improvement into the opportunity the weak model supplies, the damage exposure causes, and the cases rescued when both models failed at first.

Being an ideal bound, Δ_{ceil} needs gold answers and cannot be deployed. Its job is to **screen**: if $\Delta_{\text{ceil}} < 0$, not even a selector holding the gold answer beats the strong model, so there is no point looking for a deployable selector κ . The converse does not hold — $\Delta_{\text{ceil}} \geq 0$ says only that headroom exists in the information-theoretic sense. The identity also doubles as a bookkeeping check, matching the data for every pair we have traces for (4/4 pairs in the exploratory block, 15/15 in the main experiment, 19/19 in total); that is an algebraic check on the data, not evidence for any hypothesis. It covers only mechanisms that *choose between W and E once the artifact is in context*; deciding *whether to expose the strong model at all* is a different class with its own bound (§5.10).

3.4 Selection protocols, repair protocols, and the capability gap

Identity (1) shows that the important distinction between protocols is whether they can produce a new solution or only pick among the solutions already generated. We therefore split protocols into two classes by an operational criterion: *must the final output be an element of the generated pool?*

- **Selection:** the final output must be an element of the pool — majority voting, or selection by test, for instance. For this class, $L = 0$ by *construction*: the protocol cannot produce a new, wrong solution on a problem where the pool already contains a correct one.
- **Repair:** the protocol may generate new text after seeing the pool — a verifier that intervenes, or the exposure protocol. For this class L is no longer zero by construction and has to be measured. In our data, the 7B verifier has a very small but still non-zero L (1/300 problems, §5.5), while the exposure protocol has a much larger one (§5.9).

The other variable that matters for repair protocols is the **capability gap** between the two models: the difference between the greedy accuracy of the strong model and that of the weak model on the same benchmark, with accuracy normalised to a 0–1 scale. For the 1.5B→7B pair on MBPP, for instance, the gap is 0.244. We write g^* for the gap at which the fitted curve for Δ_{ceil} changes sign (§A). That lets us ask not only whether a coordination mechanism works, but also *under what capability distance between the models it still has the potential to create value*.

4 Experimental setup and procedure

Models. Qwen2.5-Instruct 1.5B/7B/14B/32B [23]; Llama-3.1-8B-Instruct [8]; DeepSeek-Coder-6.7B-Instruct [9]. These six models form the 15 directed pairs used in §A; the remaining blocks use Qwen 1.5B/7B/14B. Runs are spread over two hardware systems and two numeric precisions (4-bit and 16-bit); the precision difference alone reaches 3 points — the same size as the effective threshold for significance (see Statistics). Cross-hardware comparisons are therefore read by *sign* only; every quantitative comparison stays inside one combination of hardware, numeric precision and problem set.

Benchmarks. GSM8K [5] (1319 problems); MATH-500 [10] (500 problems); MBPP [1] (499 problems in the main band; 464 in the held-out band, used to test domain transfer); HumanEval [3] (164 problems). For §5.10, the key point is that MBPP and HumanEval have a *certain* check: a solution that passes the tests is direct evidence. MATH and GSM8K have no such signal.

Roles and pipeline. The four roles are *P* (planner — draft an outline), *S* (solver — solve), *V* (verifier — check, and allowed to repair), *A* (aggregator — take several candidates and pick the final answer). Combinations are written as concatenations; for example *PSVA* is the pipeline with all four roles. These letters denote *roles in the pipeline* only; *W/I/E* and *G/L/R* in §3.3 are a separate symbol set, belonging to the artifact-exposure experiment.

Aggregation methods and baselines. A *generation* is one complete solution produced by a model for one problem; it is the unit in which this report counts inference budget. The *k* generations for one problem are *independent and equal* — none precedes or depends on any other — unlike the sequential *turns* in a pipeline (the checking turn, the second turn). Given *k* generations: *maj@k* takes the most frequent answer (majority voting); *oracle@k* is correct if *at least one* generation is correct (the ideal ceiling for any way of choosing within the pool; it needs gold answers and is used only as a ceiling); *llm_agg@k* is one LLM call that reads all *k* candidates and then selects or merges. *Example:* 5 generations give the answers {7, 7, 7, 12, 15} while the gold answer is 12. *maj@5* picks 7 (wrong), while *oracle@5* is correct because one generation produced 12. On code, *exec@k* runs the tests on each candidate and takes one that passes (a certain signal); *llm@k* has an LLM review the candidates instead of running the tests (a learned signal).

Derived quantities. *v_gain* is the accuracy of a pipeline with *V* minus the accuracy of the same configuration without *V*; *A_gain* is defined the same way for *A*. AUC (area under the ROC curve) measures how well a classifier separates correct solutions from wrong ones: 0.5 is random guessing, 1.0 is perfect.

Unit conventions. Accuracy is written on a 0–1 scale; effects are written in “points” (percentage points: +14.0 points = +0.140). The “gap” is also an accuracy difference on the 0–1 scale (a gap of 0.09 ≈ 9 points). Cost comes on three scales that cannot substitute for one another: *generated characters* (a rough measure), *tokens*, and *FLOP* (parameters × tokens). Only the FLOP scale is comparable across two model sizes; cost ratios always carry a × sign (for example 0.88×) so they are not mistaken for accuracies.

Decoding. The information-flow block decodes greedily (*do_sample=False*), so the same configuration always gives the same result. Variation between folds is therefore entirely due to *different problems*, not to sampling noise.

Statistics. The role/cost block is split into five disjoint folds of 60–100 problems each. To estimate the *noise floor* of the measurement, we ran one and the same configuration on all 5 folds: *v_gain* ranged from +1.0 to +8.0 points, with a fold-to-fold standard deviation of 2.65 points. A single measurement on 100 problems therefore carries almost no information. We infer with a t-test over folds ($SE = s/\sqrt{k}$; $t_{0.975}(4) = 2.776$, i.e. an effective threshold of ~3.3 points), 95% confidence intervals (CI), and a sign test (5/5 folds with the same sign $\Rightarrow p = 0.031$ one-sided). The information-flow block uses an exact paired McNemar test on the same problem set and a paired bootstrap (10 000 resamples). Across the project’s 131 tests we expect ~6.6 false positives [2]; cases with $p \in [0.02, 0.05]$ are read cautiously as a result.

Confidence labels. *Level A* means a result measured over 5 folds with a t-test, or an experiment whose interpretation criteria were fixed in advance and which met every technical precondition we set; *level B* means a single measurement or a post-hoc stratification, used only to illustrate a mechanism. The technical preconditions check things statistics cannot see — a training adapter leaking into the control branch, say, or an answer grader that differs between the two branches; 16 of 32 runs failed a precondition and were dropped before we looked at their results.

Mid-project re-audit. We re-audited 131 quantities against one uniform statistical standard: the four headline results of the fold block (+14.0, +11.0, +5.6, +4.4 — see §5) kept their significance ($t = 3.3$ – 6.9), and the +7.7 result from the McNemar block held as well ($p = 0.0075$); 6 results were restored after we cross-checked the validity flags; 1 lost significance (it had only $k=3$ folds); and 2 pending cases were settled outright from stored fold data rather than by rerunning anything.

5 Results

This section presents the results along two lines of analysis. **The first** (from §5.1 to 5.7) works through the effect of three controls, applied one after another, on how much the agents gain from working together. **The second** (from §A to 5.11) isolates whatever value is left using the three factors G , L and κ , and closes with a discussion of training. Unless noted otherwise, the figures are at confidence level A (§4); the measurement scope of each is given in the corresponding table. A symbol lookup table is in Table 1.

5.1 The pipeline gains on GSM8K, and the measurement has a wide noise floor

Task	Solver, 1 generation	Pipeline PSVA	Difference	Generated-token ratio
GSM8K	0.632	0.744	+11.2	2.9×
MATH	0.405	0.345	−6.0	6.63×

Table 2: The full four-role pipeline against a single generation (Qwen-1.5B; one measurement over the whole set). The two middle columns are accuracy on a 0–1 scale; the *Difference* column is in percentage points.

The MATH row needs a caveat. That figure comes from a single measurement over the whole set, and two other measurements of the same quantity point the other way: a 5-fold run with 200 problems per fold gives +13.0 points with all five folds positive, and a run on MATH-500 gives +2.0 points. This is consistent with the answer-extraction problem described later in the report — the aggregator sits at the end of the four-role pipeline, and only 73–81% of its outputs produce a gradable answer. We therefore do not treat the negative figure on MATH as established, and rely on the GSM8K result.

Table 2 shows the result before any control is applied. On GSM8K the pipeline is 11.2 points ahead; on MATH it spends 6.63 times as many generated tokens and comes out 6 points behind. Two things are worth noting. *First*, the statistical test is run on a closely related but not identical quantity: over 5 folds, $PSVA - PS = +5.6$ points (CI [+3.3; +7.9]; $t = 6.89$) — the gain on GSM8K is statistically significant. *Second*, the noise-floor calibration (§4) shows that two measurements on 100 problems can differ by as much as 7 points, so effects from a single measurement are read only as exploratory signal.

The next section applies the first control: does this gain come from coordination between the roles, or simply from taking more generations?

5.2 At a large generation budget, the value is in extra independent generations rather than in coordination

Aggregation method (same 8 generations)	MATH 1.5B	MATH 7B
greedy	0.50	0.72
maj@8	0.60	0.73
llm_agg@8	0.41	0.47
oracle@8	0.73	0.85

Table 3: Aggregation methods compared at a fixed budget of 8 generations (the top row is the single-generation baseline, for reference). Cells are accuracy on a 0–1 scale.

Table 3 holds the budget fixed and varies only the *aggregation method*. Four things follow.

First, the LLM aggregator `llm_agg@8` loses to majority voting `maj@8` by 19 to 26 points at the same cost; on the 7B model it also loses to a single `greedy` generation. It breaks 21 correct answers (1.5B) and 26 (7B), while repairing only 2 and 0 incorrect ones. Across 600 traces, 65% of its turns copy the *last* candidate verbatim — closer to a positional rule than to a content-based selector; 100% of turns produce no new answer.

Second, the benefit of majority voting shrinks as the base model gets stronger. On the 1.5B model, `maj@8` improves accuracy by 10 percentage points, which is 43% of the theoretical maximum of 23 percentage points. On the 7B model the improvement is 1 percentage point, or 8% of a maximum of 13 percentage points. The `oracle@8 - maj@8` gap (13 and 12 percentage points) is reused in §5.10 as an upper bound for any selector.

Third, one control separates the effect of “coordination” from the effect of “one more generation”. When the model re-solves *without* seeing the verifier’s critique, the result equals re-solving *with* it (`rerun = loop`, both at 0.453). The gain therefore comes from the extra generation, not from the content of the feedback. (An earlier single measurement gave +20 points but did not replicate: over 5 folds it is +4.0 points.) Consistent with that, the solve-then-grade loop variant — cheaper than the full four-role pipeline — still beats it on MATH by +4.0 points (CI [+0.5; +7.5]; $t = 3.21$) at equal cost.

Fourth, the “harmful” part of the aggregator is largely a measurement artifact. Only 73–81% of aggregation turns emit an answer in the format required for grading; adding a free fallback (if the format is missing, take the verifier’s answer, with no extra model call) moves `A_gain` on MATH 1.5B from −6.4 points to +0.8 points. A separate run (different problem set, no quantization) gives +8.0 points; once the formatting bug is patched, both estimates are on the non-negative side, and the remaining difference is attributed to the different problem set and the different numeric precision.^(level B)

These four findings were all measured at a budget of eight generations, and the conclusion does not carry to small budgets. At a budget of three model calls the result goes the other way: a three-role pipeline scores 8.4 points above majority voting over three independent generations on GSM8K with the 1.5B model, and 8.5 points above it on MATH with the 7B model, with all five folds agreeing in sign in both cases. Whether coordination is worth anything therefore depends on how many generations the system is allowed to use. The conclusion above holds where that number is large.

A second control at the same three-call budget removes the role labels altogether. In the `SS_anc` configuration the solver is called three times; the second and third calls are told only that “a previous attempt answered *X*”, with no instruction to check anything. The word *check* is deliberately absent, so what is removed relative to `PSV` is the verification framing while the flow of information between passes is preserved.

	One generation	PSV	SS_anc	Difference	Tokens PSV / SS_anc
GSM8K 1.5B	0.632	0.728	0.728	0.0 (2/5)	116k / 169k
MATH 1.5B	0.330	0.380	0.360	−2.0 (3/5)	197k / 248k
GSM8K 7B	0.912	0.904	0.924	+2.0 (2/5)	115k / 179k
MATH 7B	0.500	0.590	0.480	− 11.0 (0/5)	199k / 255k

Table 4: Three sequential solver calls with an anchor (`SS_anc`) against the three-role pipeline (`PSV`) at the same call count. Accuracy on a 0–1 scale; the difference column is `SS_anc - PSV` in points, with the number of folds out of five in which `SS_anc` scored higher. Token counts are generated tokens per arm.

In three of the four cells the labels buy nothing measurable. On GSM8K at 1.5B the two configurations land on the same number to three decimals. The other two differences are 2.0 points with fold splits of 2/5 and 3/5, below the effective threshold of §4 and without a consistent direction.

The fourth cell does not follow. On MATH at 7B the named pipeline is ahead by 11.0 points and `SS_anc` loses in all five folds. On the one cell where the model is capable and the task is hard, presenting the third pass as verification does something that presenting it as another attempt does not; we did not isolate what.

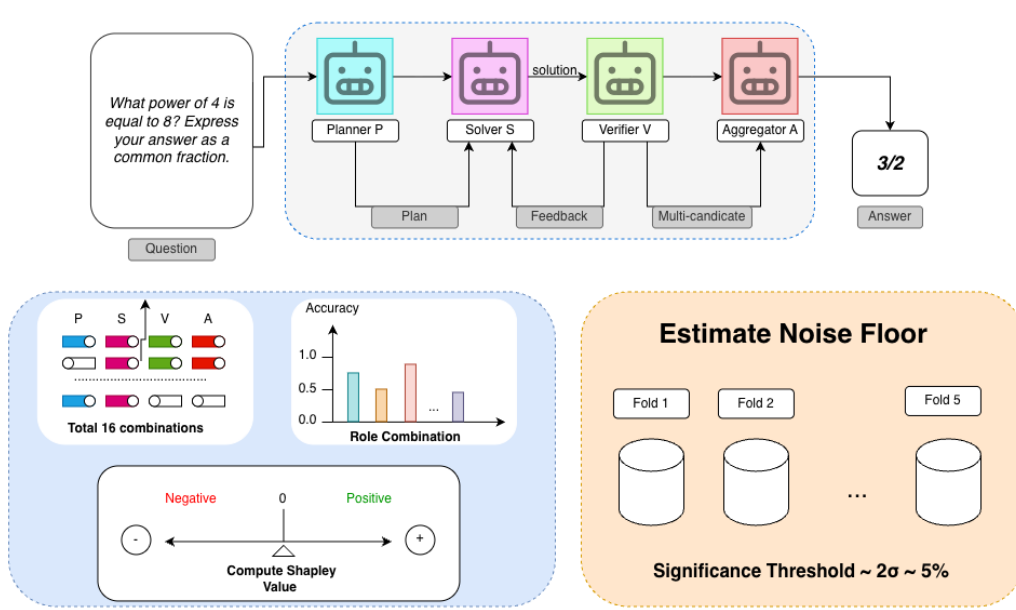


Figure 2: The four-role pipeline, the 16 on/off coalitions that were run, and the 5-fold noise estimation procedure.

Two readings therefore have to be held together. The role labels do not account for the accuracy at three of four operating points, which is consistent with the behavioural evidence in §5.4–5.6. But PSV is also never worse and is cheaper in every cell, by 25% to 56% of generated tokens, because a short plan and a critique cost less than three full solutions. The arrangement can be an efficient way to spend three calls even where the components are not performing the functions their names describe.

After the cost control, and at a large budget, the one factor with reliable value is “additional independent generations”. The next section asks: which role, and which model size, should produce those generations?

5.3 Decomposing role contributions with Shapley values

Before analysing behaviour in detail, we measure each role’s contribution with Shapley values [18]. We treat the solver as part of the fixed setup rather than as a player, and split credit among the remaining three roles. The reason is conceptual: planner, verifier and aggregator are defined only relative to an existing solution — a verifier with nothing to check is not verifying, and an aggregator with no candidates to choose between is not aggregating. The quantity being split is therefore the accuracy of the full pipeline minus the accuracy of the solver alone. We ran all $2^4 = 16$ on/off coalitions of the four roles (see Figure 2) and use the eight that contain the solver. Within that conditioned game, the contribution φ_i of a role is the mean change in accuracy when it is added to every sub-coalition — the only allocation satisfying the efficiency, symmetry and additivity axioms.

Role	GSM8K 1.5B ($N = 1319$)	MATH-500 1.5B ($N = 500$)
Planner	$-0.0188 [-0.0399; +0.0021]$	$+0.0210 [-0.0137; +0.0540]$
Verifier	$+0.0555 [+0.0430; +0.0675]$	$-0.0050 [-0.0257; +0.0150]$
Aggregator	$+0.0126 [+0.0052; +0.0202]$	$+0.0040 [-0.0100; +0.0187]$

Table 5: Shapley values of the three roles with the solver held fixed, all roles at 1.5B. The φ values are on a 0–1 scale (the share of accuracy attributed to that role). By construction the three values sum to the full pipeline’s accuracy minus the solver’s: 0.0493 on GSM8K and 0.0200 on MATH-500. 95% bootstrap CIs over questions — *no folds*, so read them together with the noise floor in §4.

Table 5 shows the following. On GSM8K, the verifier adds 0.0555 and the aggregator 0.0126, both

with confidence intervals clear of zero, while the planner’s value is negative with a confidence interval that reaches across zero. On MATH-500 none of the three intervals excludes zero, so on that benchmark none of the three roles can be separated from doing nothing. Either way the totals are small: the three roles together add 4.9 points on GSM8K and 2.0 points on MATH-500. The negative planner value on GSM8K is clearest in one pairwise comparison: $SA = 0.682 \rightarrow PSA = 0.562$, so adding the planner drops accuracy by 12 points.

Contributions also change with capability. On MATH-500, raising one role at a time to 7B while the others stay at 1.5B moves the planner from +0.0210 to +0.0686 (a change of +0.0476), the verifier from −0.0050 to +0.1454 (a change of +0.1504), and the aggregator from +0.0040 to +0.1203 (a change of +0.1163). The verifier shows the largest shift of the three: its contribution is not distinguishable from zero at 1.5B and becomes the largest of the three at 7B. This comparison is available on MATH-500 only; on GSM8K we ran the aggregator variant alone (+0.0126 $\rightarrow$ +0.1041). The verifier result sets up §5.4.

One limitation matters for reading this table: Shapley assigns credit to the *role label*, not to the *function* performed — if the planner quietly solves the problem, its work is still recorded under the label “Planner” even though the behaviour is the solver’s. The next section analyses traces to ask whether the roles actually perform their assigned functions.

5.4 The planner does not plan

The verifier contributes nothing measurable at 1.5B and becomes the largest of the three contributors at 7B. That pattern is a reason to look at what the roles actually do, rather than at what they are called. The next three subsections take the planner, the verifier and the aggregator one at a time. The same thing happens in each case: the role carries a name that describes a job, and the model does a different job.

Table 6 collects behavioural measurements from traces, and shows that on small models role specialisation is mostly nominal: the planner — despite being forbidden to compute the answer — still contains the correct answer in 34.7% of MATH problems (and in 45.3% of cases also supplies the answer format); when a plan is present, the solver merely copies it (median solution length 19 characters, against 664 without a plan). The solver row is the other half of the same observation: in 62.0% of MATH cases and 60.7% of GSM8K cases the solver produces no numeric content that was not already in the plan. A solver placed downstream of a leaking planner is transcribing rather than solving, which means the pipeline’s accuracy on those problems was decided one turn earlier than the architecture diagram suggests. Across 2,000 turns over all four cells, the aggregator produced a new and correct answer only 3 times: the aggregation role does no computation at any capability level.

Behavioural measure (traces)	GSM8K 1.5B	MATH 1.5B	GSM8K 7B	MATH 7B
Planner leaks the correct answer	14.0%	34.7%	1.0%	4.0%
Solver produces no new numbers	60.7%	62.0%	1.0%	11.0%
Solver solution length	19	344	821	1247
Verifier intervenes	32.7%	38.0%	4.0%	14.0%
Aggregator repeats the verifier’s answer	93.3%	73.3%	100%	97.0%

Table 6: Behaviour per cell (task $\times$ capability, $n = 100 - 150$ per cell, measured on traces).

One qualifier covers this subsection and the two that follow, and we state it here once rather than three times. On the 7B model the planner leak, the solver’s copying and the verifier’s intervention rate all fall to a few per cent, so everything in §5.4–5.6 is a claim about *small* models. Two things do not soften with capability. First, at 7B the pipeline loses to the single solver (§5.8), so the capability level at which the roles behave themselves is also the level at which the architecture stops paying. Second, the aggregator is unchanged: the count of 3 new and correct answers is taken over all four cells, not over the small models alone.

The role boundary is not enforceable by instruction. We ran the planner under three prompts over 5 folds, holding the rest of the pipeline fixed: `HIDE` (“Give a concise numbered plan. Do NOT compute the

final answer.”), `FREE` (“Give a concise numbered plan.”) and `ASK` (“...and compute the final answer at the end.”). Table 7 gives the maths results and Table 8 the code results. Told in the prompt not to compute the answer, the planner computes it anyway in 33.3% of GSM8K problems and 21.3% of MATH problems. Told not to write code, it writes code in 53.7% of HumanEval problems. Told to cross the boundary, it crosses it every time: 100.0% of `ASK` plans contain code. The prohibition moves the rate at which the boundary is crossed; it does not hold the boundary.

Planner prompt	GSM8K leak	GSM8K solver	MATH leak	MATH solver
<code>HIDE</code> (“do NOT compute the answer”)	33.3%	0.6700	21.3%	0.4075
<code>FREE</code> (no prohibition)	48.5%	0.7025	27.8%	0.4450
<code>ASK</code> (“compute the answer”)	46.3%	—	25.0%	—

Table 7: The planner-prompt study on maths (5 folds per condition). “Leak” = the plan already contains the correct final answer; “solver” = accuracy of the solver run downstream of that plan. The solver was not run downstream of the `ASK` condition.

These leakage rates are not the same measurement as the planner row of Table 6, and they do not agree with it in size: the trace measurement inside the full pipeline gives 14.0% on GSM8K and 34.7% on MATH, the prompt study gives 33.3% and 21.3%. The two use different problem samples and different measurement setups, and we make no attempt to reconcile them. Both should also be read as upper bounds rather than as counts of deliberate computation: each detects a leak by taking the last number in the plan and comparing it with the gold answer, so a plan that merely restates a quantity from the problem can be scored as leaking. The code measurement does not share this weakness — it looks for a code fence or a `def` statement, which a plan does not produce by accident — and it is the one the argument rests on. The maths figures corroborate the direction; they are not precise rates.

A related control points the same way. When prompts are permuted across positions, accuracy is indistinguishable: on GSM8K $0.660 = 0.660$ (the `sol` configuration reaches 0.673); on MATH `swap-normal` = +5.3 points but $t = 1.84$, CI $[-2.7; +13.4]$ — not statistically significant. The wide CI does not let us rule out a medium-sized effect, so we record only that there is no evidence that “role identity” affects the outcome.

Relaxing the instruction changes accuracy in opposite directions by domain. On maths, dropping the line “Do NOT compute the final answer” raises leakage and *raises* the downstream solver by +3.8 points on MATH and +3.3 on GSM8K. On code, dropping the same line also raises leakage — 53.7% to 73.8% — and *lowers* accuracy by 5 points, from 0.4312 to 0.3812. The reading we take from this, and it is a reading of the numbers rather than a demonstrated mechanism, is that the two domains differ in what a leak is worth. A leaked arithmetic answer is a complete, checkable object: it functions as a free second attempt at the problem, and the solver can arrive at it independently or adopt it. A leaked partial implementation is not a second attempt; it is a starting state, and the solver continues from it. When that start is bad, the solver inherits the mistake instead of getting a second shot at avoiding it. On maths the instruction only hides the answer rather than preventing it from being computed, and hiding it does harm; on code the same relaxation costs accuracy. We have not run an experiment that separates “free second attempt” from “anchor”, so this stays a reading.

One detail in Table 7 does not fit any account we have: on both maths tasks the highest leak rate is `FREE`, not `ASK`, even though `ASK` instructs the planner to produce the answer. We record it and do not build on it.

On code the planner is net harmful. The code study includes a condition the maths study does not: no planner at all. Table 8 shows that removing the planner entirely is the best of the four conditions. Adding a planner costs 10.6 points against running the solver with no plan — 0.5375 against 0.4312 for the best planner condition — and `P_hide` is at or below `NoP` in 5 of 5 folds. That is comfortably past the ~3.3-point effective threshold set in §4, and no fold contradicts it. On this benchmark the planning role is not a

small positive that fails to clear the noise; it is a cost.

Planner condition (HumanEval)	Accuracy	Plan contains code
No planner (NoP)	0.5375	—
Planner, <code>HIDE</code> (“do NOT write code”)	0.4312	53.7%
Planner, <code>FREE</code> (no prohibition)	0.3812	73.8%
Planner, <code>ASK</code> (“write the code”)	0.4437	100.0%

Table 8: The planner-prompt study on HumanEval, 5 folds $\times$ 32 problems (160 of the benchmark’s 164). Every planner condition is below the no-planner condition. `P_hide` is at or below `NoP` in 5/5 folds; `free` minus `hide` is negative in 4/5 folds.

How much of this to believe. Three caveats, all of which apply to Table 8. (i) A fold here holds $n = 32$ problems, which is smaller than the 60–100 problems per fold used elsewhere in the report; the noise floor calibrated in §4 is therefore a lower bound on the fold-to-fold spread of these numbers, not a fair estimate of it. (ii) The `FREE`-versus-`HIDE` difference on code is 5 points with the same sign in 4 of 5 folds, which puts it near the noise floor; we treat the *direction* of that comparison as the finding and not its size. (iii) The one result here we regard as solid is the 10.6-point cost of adding a planner at all, which holds in 5 of 5 folds and is three times the effective threshold.

What this does to the measurement. The consequence is not that planning is worthless. It is that a pipeline containing a component labelled “planner” cannot be used to measure what planning contributes, because the component is not doing the job the label describes. When the plan already carries the answer, the “planner + solver” number is a number about sampling the model twice and letting the second sample copy the first. When the plan carries a half-written function, it is a number about how much a bad prefix costs. Neither is a measurement of decomposition. Reading a planner’s contribution off such a pipeline attributes to coordination something produced by leakage.

5.5 The verifier does not verify at its own capability

The verifier is the second instance of the same pattern. At equal capability its interventions are close to chance, so what the checking role buys is not verification — it is a larger model.

Three independent measurements. (i) When it intervenes, the 1.5B verifier is right only 56–59% of the time — close to chance; the 7B verifier is right 98% of the time. (ii) An error-injection experiment — altering the result of one computation step in a gold solution chain and then asking the model “is there a calculation error?” — shows the 1.5B model answering “no error” on close to 100% of problems, i.e. blind to the injected error; the 7B model passes the validity gate and separates clean from altered problems (+0.651 on the stratum of problems it can solve on its own; the stratum it *cannot* solve on its own has only $n = 9$, so the question of whether checking is separable from solving remains open). (iii) The `V_gain` grid (Table 9) shows that the checking turn pays off only within a narrow band.

<code>V_gain</code> (points; min–max across folds)	GSM8K	MATH
1.5B	+4.4 [+1; +8], 5/5 positive	+1.4 [−1; +4]
7B	+1.0 [−3; +5]	+4.4 [+2; +8], 5/5 positive

Table 9: `V_gain` by task $\times$ capability (5 folds per cell). Ordering the four cells by solver accuracy — 0.402 (overwhelmed); 0.598 (pays off); 0.668 (pays off); 0.884 (saturated) — the checking turn pays off only in the solver-accuracy band ≈ 0.60 – 0.67 : below it the verifier is overwhelmed, above it there are no errors left to fix.

The clearest driver is the capability of the model doing the checking. On the same 300 MATH problems (5 folds $\times$ 60), Table 10 shows:

Checking turn	Effect (points)	95% CI (points)	Repaired : Broken
Verifier 7B (solver 1.5B)	+14.0	[+7.4; +20.6]	43 : 1
Verifier 1.5B (same size)	+3.0	CI contains 0	15 : 6
Portion attributable to the larger model	+11.0	[+3.9; +18.1]	—

Table 10: The asymmetric effect of capability at the checking turn on MATH. “Repaired : Broken” = incorrect answers turned correct against correct answers turned incorrect; the same-size verifier breaks 6 problems, while the 7B verifier breaks only 1 over the same 300 problems.

Two columns of that table carry the point. The same-size verifier’s effect is +3.0 points with a confidence interval that contains zero, and its repair-to-break ratio is 15 : 6 — it breaks roughly one correct answer for every two and a half it fixes. The larger verifier reaches 43 : 1. Of the +14.0 points, +11.0 (CI [+3.9; +18.1]) is attributable to model size alone. Run at its own capability, the role is not adding a check; it is adding a coin flip that occasionally damages a correct answer.

The mechanism from the traces: when the verifier intervenes, it reuses the solver’s intermediate numbers at roughly one-fifth the rate it does when it agrees (0.20–0.23 against 0.96). Intervening is therefore closer to re-solving than to step-by-step checking, and in effect the checking turn is equivalent to one extra generation. This fits the hypothesis that the +14.0 points come mainly from adding one generation from a stronger model. The same picture appears elsewhere: `V_gain` is statistically significant on MATH 7B (+4.4 points; CI [+1.5; +7.3]) but not on MATH 1.5B (+1.4; $p = 0.235$). A note for §3.4: the verifier *generates a new chain*, so it belongs to the repair class — its measured L is $1/300 \approx 0.003$, very small but structurally not zero.

5.6 The aggregator does not aggregate

The aggregator is the third instance. It selects; it does not aggregate.

The measurement is the one already given above: across 2,000 turns over all four cells, the aggregator produced a new and correct answer only 3 times. Whatever the aggregation turn does, it is not computation — it returns one of the candidates it was handed. Table 6 shows how narrow the selection is in practice: the aggregator repeats the verifier’s answer in 93.3% of GSM8K turns and 73.3% of MATH turns at 1.5B, and in 100% and 97.0% of turns at 7B. On the question of *how* it selects — and in particular the finding that the choice tracks position in the candidate list more than content — see §5.2, which also shows that part of the measured harm from the aggregation turn is a formatting artifact rather than a bad selection.

The name of the role promises a combination step: read several candidate solutions, reconcile them, produce the answer they jointly support. Counting votes does that, and does it without a model call. The LLM aggregation turn does not. It picks one candidate and passes it on, and the 3-in-2,000 count sets an upper bound on how often it does anything else. That is the same failure as in the two subsections above, in its cleanest form: the role name describes an operation, the operation is not performed, and the pipeline number gets attributed to it anyway.

The +14.0 figure of §5.5 is measured against the weak model. §5.8 applies the second and first controls together: comparison against a strong-model baseline, with token cost taken into account.

5.7 Stratifying by the reach of the aggregation method

Table 11 splits the 150 problems by “how many correct solutions the pool contains”. Two fixed quantities need to be kept apart. By the structure of the pool, 57% of the problems (0/5 and 5/5) cannot show any effect under any aggregation method — there is either nothing to select or nothing left to improve. For majority voting in particular, adding the 1/5 stratum (a single vote cannot win a majority) brings the total to 70%. On the reachable strata, the effect of voting is +21.5 points (strata 1–4/5) or +31.1 points (strata 2–4/5); the full-set average dilutes this to +9.3 — a dilution factor of 2.3–3.3 depending on how the strata are grouped. The strata are defined after the fact, using the generation results themselves, so this is an

Correct generations /5	n	Solver	maj@5	Δ (points)
0/5 — too hard	48 (32%)	0.000	0.000	0
1/5	20 (13%)	0.000	0.000	0
2/5	15	0.333	0.600	+26.7
3/5	12	0.583	1.000	+41.7
4/5	18	0.722	1.000	+27.8
5/5 — too easy	37 (25%)	1.000	1.000	0

Table 11: Stratification by the number of correct generations out of 5 (MATH 1.5B, $n = 150$). The Δ column is the accuracy difference between maj@5 and the Solver.

interpretive tool, not a deployable baseline. A genuine +10-point intervention that only reaches strata 2–4/5 (30% of the problems) would show up as +3.0 points over the full set — below the noise floor.

The three controls therefore pull in opposite directions: a missing denominator $\Rightarrow$ underestimate; a missing cost or baseline correction $\Rightarrow$ overestimate. Self-declared limitation: the headline effects in §5.4–5.8 have not been re-stratified this way (we lack per-problem data for each stratum); their full-set numbers are therefore low estimates of the effect on the reachable strata.

End of the first thread. The picture after three controls: the only value that is reliably positive is the extra generation; configurations in which models repair each other’s work lose to the strong model alone. Act II asks where the value goes, and whether any rule predicts it in advance.

5.8 The strong model alone remains the benchmark on both accuracy and cost

Configuration	GSM8K	MATH	7B tokens (GSM8K)	7B tokens (MATH)
S1.5B + V7B	0.810	0.563	105k	119k
S7B alone	0.910	0.593	120k	152k
S7B + V7B	0.900	0.670	205k	261k

Table 12: Comparison against the strong-model baseline: configurations that include a verification pass, measured against the strong model run alone (5 folds). The token columns count only tokens generated by the 7B model — they cannot be compared directly across configurations built from different model sizes; the correction is given in Table 13. The two middle columns are accuracy (0–1 scale), the last two are token counts (thousands).

Table 12 moves the baseline from “the weak model” to “the strong model alone”, and the picture changes. The asymmetric configuration S1.5B+V7B — the same configuration that gave +14.0 points in Table 10 — is *worse* than S7B alone by 10 points on GSM8K, and only matches it on MATH. The cost column needs two corrections: (a) the raw token column ignores the tokens spent by the 1.5B solver; (b) a 7B token costs roughly 4.7–4.9 times more than a 1.5B token in FLOPs. Table 13 rescales cost onto the FLOP axis:

Cost ratio (asymmetric / S7B)	Raw 7B tokens	FLOP-weighted
GSM8K	$0.88\times$ (“12% cheaper”)	1.00–1.05 $\times$
MATH	$0.78\times$ (“22% cheaper”)	0.92–0.96 $\times$

Table 13: Correcting the unit price of a token (sensitivity swept over characters-per-token from 3.0 to 4.0). Prefill is not counted — the 7B model has to *read* the 1.5B solution — so the correction still favours the asymmetric configuration.

The conclusion: on GSM8K the asymmetric configuration is dominated outright (10 points worse, at equivalent FLOP cost); on MATH it saves a marginal 4–8% at equivalent accuracy. The strong model alone sits on the Pareto frontier for both tasks.

The one bright spot in Table 12 that is distinguishable from zero comes from the same-size configuration: adding V7B to S7B on MATH gives +7.7 points ($p = 0.0075$; McNemar), at $1.7\times$ the token cost. In the notation of §3.3 this is the case $\text{acc}(E) > \text{acc}(I)$ — and it occurs at a *capability gap of zero*. The same configuration on GSM8K spends $1.7\times$ the cost to lose 1 point — the benefit of a same-size verification pass depends on the task. We return to this data point in §A, where it is the only evidence supporting the positive half of the capability-gap rule.

Because the same model pair on the same family of problems appears with several different values across this report, Table 14 lists them side by side to avoid confusion:

Value	Quantity	Baseline	Setup	Section
+14.0 points	verification-pass effect	weak model (PS)	verification prompt, 300 problems	§5.5
−3.0 points	configuration accuracy	strong model (S7B)	as above, 5 folds	§5.8
−12.4 points	$\Delta_{\text{real}} = \text{acc}(E) - \text{acc}(I)$	strong model	<i>solve</i> prompt, artifact shown, $n = 500$	§5.9
−16.6 points	Δ_{ceil} (ideal ceiling)	strong model	repair prompt, bootstrap	§A

Table 14: The same 1.5B→7B pair on the maths domain, four different quantities. The change of sign between the first row and the three below it is the central phenomenon itself: changing the baseline changes the sign of the effect.

One control is still missing: the denominator. Every number above is an average over the whole set — including the problems that no mechanism could have rescued.

5.9 Making the checker stronger: what it buys and what it costs

The three preceding subsections all measure a single model size, and they all point the same way: the planner solves, the verifier at its own size intervenes at close to chance, and the aggregator selects rather than aggregates. The obvious remedy is to stop asking a model to check work at its own capability level, and to put a stronger model in the checking seat instead. This subsection asks what that move buys and what it costs.

What it buys is already in the results above. On the checking turn, §5.5 reports that when it intervenes, the 1.5B verifier is right only 56–59% of the time — close to chance; the 7B verifier is right 98% of the time. Table 10 puts the asymmetric configuration — a 7B verifier over a 1.5B solver, on the same 300 MATH problems — at +14.0 points. A stronger checker does do work that a same-size checker does not.

What it costs is measured here, in the cleanest experiment in the report (pre-registered; we chose MATH-500 to change domain relative to the original exploratory observation on MBPP). The design is shown in Figure 1: one problem set of 500 MATH problems and two branches, I and E . Both branches use the same solve instruction, the same generation budget and the same strong model; they are identical in every other respect, differing only in whether the context carries W ’s artifact. Results are stratified by artifact content and reported in Table 15.

Stratum	MBPP 11–510 ^(level B)	MBPP 511–974 ^(level B)	MATH-500 (level A)
artifact wrong	−19.0	−19.3	−27.2 ($p \approx 0$; $n = 261$)
artifact correct	+6.4	+2.5	+3.8 ($p = 0.012$; $n = 239$)

Table 15: Exposure effect $\text{acc}(E) - \text{acc}(I)$ (units: percentage points), stratified by artifact content. The two MBPP columns are post hoc stratifications (level B); the MATH column is the pre-registered experiment (level A).

On the wrong-artifact stratum, the accuracy of the strong model drops from 46.4% to 19.2% — on exactly the problems it was already solving nearly half the time. A correct artifact, by contrast, improves accuracy. Weighting the two strata reproduces $\Delta_{\text{real}} = -12.4$ points. The mechanism follows: D is not a penalty for seeing an artifact at all, but a penalty for seeing wrong content. Since the design carries no repair instruction, the mere presence of wrong content in the context is enough to produce the negative

effect. (A matched control for the pure effect of context length has not been run; the closest control — inserting irrelevant context in the checking setup^(level B) — gives -3.6 points, so pure context noise is mildly harmful, seven to eight times smaller than the wrong-content penalty.) This matches the low reuse rate under intervention in §5.5: the strong model rarely copies wrong content at the surface, yet is still anchored by it — the influence travels through the choice of approach more than through quotation.

Table 16 re-scores all 500 problems by the state pair (W correct/wrong) $\times$ (I correct/wrong), and records the share of correct E answers in each cell, to answer whether repair helps when the strong model is wrong.

Stratum	n	Share of E correct (%)	
W correct, I correct	228	99.6	
W correct, I wrong	11	90.9	
W wrong, I correct (risk zone)	121	36.4	(destroys 63.6%)
Both wrong	140	4.3	

Table 16: 2×2 stratification by the states of W and I on MATH ($n = 500$), with the share of correct E answers in each stratum.

Three main observations follow from Table 16. First, the repair protocol acts as a conduit, not an error corrector: it almost never produces a new solution when both models miss ($4.3\% = 6/140$), it destroys at scale in the risk zone ($63.6\% = 77/121$), and it passes through W ’s correct answer when there is one (10 of 11 problems — the stratum is too small for a tight estimate). Second, for this pair the exploitable stratum is 11 times smaller than the risk zone (11 problems against 121); that ratio scales with the capability gap, but we did not measure it at other gap sizes. Third, the strategy “keep W when W is right” is equivalent to the CEIL oracle. Even when handed that oracle, the gain is only $+0.4$ points (2 more correct out of 500), because E already retains 99.6% and 90.9% in the two corresponding strata. Even at $\text{CEIL} = 0.578$, the value stays below $I = 0.698$. Within the class of asymmetric repair protocols we measured, no configuration reached $\text{acc}(E) > \text{acc}(I)$ (0/15 MBPP pairs, best -3.0 points; the MATH pairs are in Table 19); the only case of $E > I$ is the same-size configuration in §5.8. The problem is not that the system “forgets to keep W ”, it is that E destroys the risk zone. Among the options we tried (the two oracle gates in §5.10), only keeping the artifact out of the context protected that zone.

The two halves therefore constrain each other. The stronger checker is worth $+14.0$ points at the checking turn (Table 10), and it costs -27.2 points on the stratum where the weak model is wrong, once it is shown that weak model’s work. What survives is a narrow prescription: bring in the stronger model, but do not put the weaker model’s solution in front of it. That rules out the generate-then-repair arrangement which is the reason most of these pipelines are built in the first place.

5.10 From verification signal to selection and routing

A certain signal. On HumanEval, with the same model and the same 4 generations, differing only in the source of the check, the results are in Table 17.

Condition	greedy	exec3	llm3	exec3 destroyed	llm3 destroyed
HE 1.5B (run 1)	0.5375	0.6000	0.4812	0.0	2.8
HE 1.5B (run 2)	0.5625	0.6438	0.4375	0.0	4.6
HE 7B (run 1)	0.8000	0.8812	0.7812	0.0	2.6
HE 7B (run 2)	0.7938	0.9000	0.7438	0.0	3.2

Table 17: Selection by running tests (exec3) compared with LLM self-review (llm3). Each model was measured twice; the two runs per model were collected on two different hardware setups, but the table does not record which run corresponds to which setup. The spread between the two runs of a model is about 0.03, smaller than the effect of interest, so the conclusions about sign still hold. The “destroyed” columns give the number of problems broken per fold.

`exec3` destroyed nothing in 20 of 20 folds and matched `oracle@4` exactly in the two cells where that figure was recorded (0.6438 and 0.8812). `llm3`, by contrast, destroyed solutions in all 20 folds. The honest baseline is greedy; majority voting is harmful on code, because long solutions rarely coincide verbatim. The difference `exec3` – greedy ranges from +6.3 to +10.6 points across the four conditions. One condition on the application side matters: when the base model is saturated (GSM8K 7B, solver 0.916), an execution checker with accuracy 0.837 still gives a net negative effect (26 destroyed / 6 fixed). “Having a certain checker” is therefore only worth something while headroom remains.

A learned signal. The classifier was trained on injected errors (take a correct solution and change one digit in the answer) and then used to detect real errors. This is also the test “change one number in the answer — does the verifier notice?”. Results are in Table 18.

Model	Separation on injected errors	Separation on real errors	AUC	Δ accuracy
1.5B	−0.012	+0.195	0.528	−0.8 points (1/5 folds positive)
7B	+0.573	+0.693	0.893	+2.4 points (2/5 folds positive)

Table 18: Effectiveness of the learned error classifier. The first two columns give the separation in classifier score between correct and wrong solutions, measured on injected and on real errors. At 1.5B, AUC 0.528 $\approx$ random guessing: change one digit in the answer and the 1.5B verifier does not detect it.

At 7B the measured signal is good (AUC = 0.893; separation is even better on real errors than on injected ones, so we would need to rule out the classifier latching onto surface features such as length; that control has not been run). But when the signal is used for selection, the improvement is only +2.4 points, and only 2 of 5 folds are positive. By our own sign test, this gain is not distinguishable from 0. A measurable signal therefore does not automatically become a usable one, which supports the upper-bound statement below.

A combined statement. Placed side by side: on code the `oracle@k` – `maj@k` gap is wide and `exec3` takes nearly all of it (it matches `oracle@4` exactly and destroys nothing); on math the gap is 12–13 points (Table 3) but the best learned signal takes only +2.4 points.

The gap `oracle@k` – `maj@k` is an upper bound on the value of any checker over the same pool. A certain signal (running tests) takes nearly all of that bound; the learned signal in our study takes only a small part of it, even though its error measurement is of high quality. The bottleneck in the math domain is therefore twofold: no certain signal, and a poor pool. Among the problems where `maj@5` is wrong, 65% (48/74, computed from Table 11) are problems whose pool does not contain a single correct candidate, so no selector could rescue them.

Routing. Both results on “choosing when an extra call is needed” are level B (measured once, $n = 300$). The consensus router (run S and V , call A only when the two roles disagree) on GSM8K keeps almost all of the pipeline’s benefit at 77% of the cost (0.720 against 0.723, at 2.32 against 3 calls); on MATH it reaches 0.413, equal to the solver alone and therefore of no benefit. The mechanism behind this is consistent with §5.9: the share of problems the weak model gets wrong (that is, the share of wrong artifacts in the context) is about 33% on GSM8K against 59% on MATH; when the two roles disagree, A rescues 45.4% of cases on GSM8K but only 25.0% on MATH. The observation that “routing saves cost exactly where saving is least needed” therefore stays at level B.

Two ceilings for the exposure gate (computed on the data of §5.9): a perfect artifact gate (show only correct artifacts — an oracle) reaches +1.8 points over I ; a real classifier would need about 88–89% accuracy to break even (break-even $\approx 27.2/(27.2+3.8)$ from the two effects in Table 15). A gate that knows when I is wrong (a stronger oracle) reaches +3.2 points, but it requires detecting the strong model’s own errors — exactly the kind of signal we have just shown to be measurable but not usable. The implication

is that improving the gate has a low ceiling; the default configuration, which keeps the artifact out of the context, has a higher expected value.

5.11 Role training and the shortcuts of the objective function

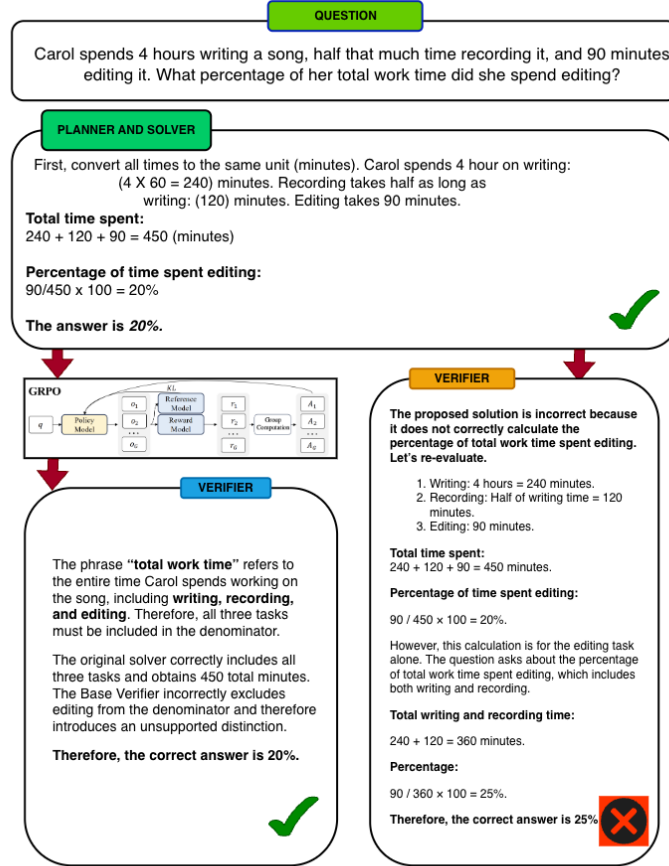


Figure 3: A concrete case on GSM8K: the solver answers correctly (20%), the original verifier “fixes” it into a wrong answer (25%) — exactly the risk-zone destruction of §5.9; the verifier after GRPO keeps the correct answer, but it achieves that mainly by intervening less, not by checking better. Figure reused from the group’s Shapley block report.

We tried seven methods for training and optimising roles. None met the project standard (effect above threshold and 5/5 folds with the same sign) on held-out data, and every failure traced back to the model finding a shortcut in the objective function instead of learning the target skill.

First. GRPO verifier with reward +1 for a correct fix, −1 for a destruction, 0 for silence. After training, precision rises from 0.70–0.90 to 1.00 (5/5 folds), but V_{gain} falls from +6.8 to +4.4 points, and the number of interventions falls from 20.2 to 8.4 per 100 problems. The shortcut is “free silence”: perfect precision is obtained by the model speaking half as often.

Second. GRPO on the final answer (patching the silence hole) reaches +1.8 points, below the +2.0 threshold fixed in advance. The shortcut is mimicking the solver: median output length falls from 480 to 19 characters, and the rate of agreeing when the solver is wrong rises from 0.50 to 0.64.

Third. GRPO with information asymmetry (solver 0.5B, verifier 1.5B; pre-registered, with a full I branch) measures two baselines at once: against the weak model it gives +17.0 points (5/5 folds positive); against the checking model itself solving on its own it gives −10.4 points (5/5 folds negative). RL

recovers only 38% of the exposure damage; 54% of the broken problems produce a third answer that follows neither model.

Fourth. ORPO aggregator (learning from preference pairs) reaches +3.3 points on MATH (3/5 folds, below the noise floor) and −2.7 points on GSM8K cross-task. The shortcut is amplifying the habit of copying the last candidate (from 0.71 to 0.81) and learning to “solve it again” instead of to select.

Fifth. Credit-RL (rewarding roles by their marginal Shapley contribution) improves no role by the 5/5-fold standard. The planner collapses to empty plans on 200 of 200 problems; the verifier makes 23 fixes and 24 destructions.

Sixth. Credit-RL stage 2 (a shortcut-resistant design) gives V-COND +3.0 points (4/5 folds positive) on GSM8K, the only positive variant, but it does not carry across domains: on MATH it gives −7.0 points (5/5 folds negative).

Seventh. MAPoRL co-training of S/V/A leaves the result unchanged ($0.690 \rightarrow 0.690$), because the aggregator was bottlenecked throughout training.

Each method had either a design fixed in advance or an effect gate; per-run details and corrections are in `docs/`. The eighth method — the supervised classifier — was presented in Table 18.

Figure 3 illustrates one concrete case. The information-asymmetry experiment (method three) is the one worth singling out, because it measured two baselines on the same run. Against the weak model, the effect is +17.0 points (5/5 folds positive); against the checking model itself when it does not see the artifact, the effect is −10.4 points (5/5 folds negative). This is a training-based replication of the sign flip under a change of baseline (§6): RL narrows the exposure damage (recovering 38%) but does not reverse the sign of Δ_{real} . The general conclusion across all these methods connects straight to §5.10: when the reward signal is itself only a learned signal, optimisation finds the shortcut in that signal faster than it finds real checking ability.

6 Synthesis: the roles are nominal, and the obvious remedy is not free

Read together, the measurements taken at a single model size describe a pipeline whose parts do not keep the shape they were given. Six of them point the same way:

- With the solver held fixed, the other three roles add 4.9 points on GSM8K; on MATH-500 none of the three confidence intervals excludes zero (§5.3).
- Re-solving *without* reading the verifier’s critique gives the same accuracy as re-solving with it (`rerun = loop`, both at 0.453). The content of the feedback contributes nothing beyond the extra generation that carries it (§5.2).
- At a fixed budget of eight generations, the LLM aggregator loses to counting votes by 19 to 26 points (§5.2).
- A planner told not to write code writes it in 53.7% of HumanEval problems; on maths the plan’s final number already matches the gold answer in 33.3% of GSM8K problems; told not to write code, it writes code in 53.7% of HumanEval problems. On HumanEval a planner is net harmful: 0.5375 with no planner against 0.4312 with one, with the no-planner condition at or above the planner condition on 5 of 5 folds (§5.4).
- A verifier at the solver’s own size is right 56–59% of the time when it intervenes — close to chance (§5.5).
- Seven attempts to train a role into its assigned function produced seven shortcuts in the objective instead (§5.11).

The reading we take from this is that what a four-role pipeline gains at a single model size comes mostly from sampling the model more than once, not from the division of labour. The role labels describe the prompts, not the behaviour, and a quantity attributed to a role is in large part a measurement of one more generation.

Making the checker stronger, and the two baselines. If the roles do not work at equal capability, the obvious remedy is to give the checking turn to a stronger model. That move does produce a large number: §5.5 records a capability gap of 0.244 together with a gain of +14.0 points. But §A shows that a large gap drives the maximum attainable improvement Δ_{ceil} negative.

The two do not conflict. They are two measurements against two different baselines, and the capability gap moves the two baselines in opposite directions (Figure 4):

- *Measured against the weak model W* (the baseline most studies report against): the checking role amounts to one more independent solution attempt by a stronger model (§5.5). What the number reflects is the intrinsic capability difference plus the advantage of an extra generation, so the gain grows with the capability gap (from +3.0 points at a gap of 0 to +14.0 points at a gap of 0.244).
- *Measured against the strong model run independently I* (the baseline that matters in deployment): the net value follows $G - L + R$. The improvement opportunity G shrinks as the gap widens ($\beta_1 = -0.19$, §A), while the loss L grows (on MATH at a gap of 0.244, L is about 7 times G in magnitude). The combined value therefore falls, and the only positive configuration appears when the two models are of comparable capability (+7.7 points at a gap of 0, §5.8).

What we are seeing here is not a property of the coordination mechanism but of the baseline we choose to compare against. A capability gap inflates the apparent effect relative to the weak model and erodes the real added value relative to the strong model.

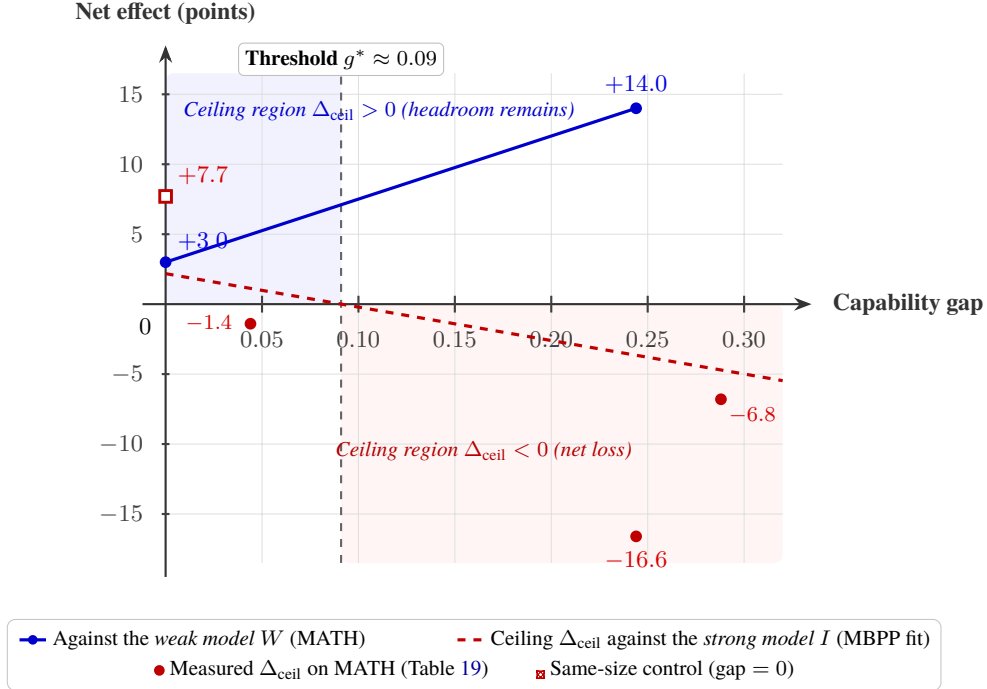


Figure 4: Two baselines, two directions of change. The horizontal axis is the capability gap. The effect measured against the *weak model* W (solid blue line) rises with the gap; the ceiling on net value Δ_{ceil} against the *strong model* I (dashed red line: fit over 15 MBPP pairs; red dots: 3 MATH measurements) falls quickly and turns negative past the critical threshold $g^* \approx 0.09$, where the exposure loss L outweighs the improvement opportunity G .

Five experimental results and one structural consequence support this reading of the two baselines: (1) *structurally*, the selection protocol removes the loss entirely ($L = 0$) by construction — this follows from how the protocol is defined and is not a measurement; (2) *in magnitude*, the loss L outweighs the improvement opportunity G ($L \approx 7G$) at a large capability gap on MATH — in the 2×2 stratified table (Table 16) L covers 77 of the 500 problems against 11 for G . This ratio between the two terms is a different quantity from the stratum-size ratio reported in the same table, where the exploitable stratum is 11 times smaller than the at-risk stratum (11 problems against 121); the two share the numeral 11 but are not the same comparison; (3) *mechanistically*, the loss comes directly from exposure to a wrong artifact, which we confirmed in a pre-registered experiment on two task domains; (4) *at the theoretical ceiling*, even ideal routing yields only a modest +1.8 points against a decline of −12.4 points; (5) *on generality*, the rule established on MBPP holds on 2 of the 3 model pairs in the MATH set; and (6) *on reproducibility through training*, reinforcement learning under information asymmetry records a gain of +17.0 points against the weak model and a drop of −10.4 points against the strong model at the same time, on 5 of 5 folds (§5.11). Independent reports that multi-agent debate underperforms self-consistency on small models [4] point in the same direction.

The cost of the remedy is therefore specific, and it is measurable. Making the checker stronger means placing the weak model’s solution in the strong model’s context, and that is exactly the configuration of §5.9: on the stratum where the weak model’s solution is wrong ($n = 261$), the strong model’s accuracy falls by 27.2 points, from 46.4% to 19.2%. On the other stratum a correct artifact is worth +3.8 points, and weighting the two reproduces $\Delta_{\text{real}} = -12.4$ points. The exposure-loss mechanism accounts for the erosion in the figure above: the wider the capability gap, the more likely the artifact is wrong, and the more often the strong model is steered off a problem it could have solved on its own. The remedy for roles that do not work at equal capability thus carries a loss of its own, and that loss grows with the same quantity the remedy relies on.

System design recommendations (for the 1.5B–32B range on reasoning tasks). Four conditions decide whether a multi-agent arrangement is worth building; failing any one of them leads to the same recommendation, which is to use the strong model alone:

- **Role boundaries stated in a prompt:** do not assume a role performs the function its instruction names. A planner told not to write code wrote it in 53.7% of HumanEval problems; on maths the plan’s final number already matched the gold answer in 33.3% of GSM8K problems; told not to write code, it wrote code in 53.7% of HumanEval problems. Measure the behaviour before crediting the label. If the components solve in every position, what the arrangement buys is repeated sampling of one model, and that budget is better spent on the strong model alone.
- **Domains with a deterministic verification signal** *where the model has not saturated:* the best option is to generate candidates independently and select on execution results. This approaches the oracle ceiling without damaging solutions that were already correct. Once the base model has saturated on accuracy, adding a checker can lower overall performance (we recorded 26 cases broken against 6 cases fixed on GSM8K 7B).
- **Domains without a deterministic verification signal:** majority voting captures most of the available headroom on small models (43% on MATH 1.5B), but the effect drops on larger models (8% on 7B) and can backfire on code generation. An LLM aggregation role gives no consistent advantage over mechanical voting. Learned classification signals separate well on the held-out set but have not improved end-task accuracy.
- **Cross-model coordination:** handing a small model’s artifact to a large model for repair gives a negative effect or one that is not distinguishable from zero, particularly once the capability gap exceeds ~ 0.09 (on MBPP). The checker has to be stronger than what it checks, and here it is the checked artifact that drags the checker down. In this case the option that is best on both accuracy and cost is to call the strong model alone.

Even when all four conditions hold, an unverified solution from a weak model should not be placed in a strong model’s context.

7 Conclusion and limitations

The practical value of multi-agent LLM systems does not reduce to a single verdict. It depends systematically on three measurable quantities: (1) the improvement opportunity G , which is set by the capability gap and tends to narrow as that gap widens; (2) the exploitability κ , which depends on whether the verification signal is deterministic rather than on the estimated quality of the classifier; and (3) the exposure loss L , which arises from contact with a wrong artifact, and where repair protocols act mainly as a conduit for the answer rather than as a genuine fix to the reasoning (§5.9). The capability gap between models both raises the apparent improvement relative to the weak model W and erodes the net value relative to the strong model I . In domains with a deterministic verification mechanism, multi-agent coordination approaches the oracle ceiling without damaging solutions that were already correct — but this holds only while the base model is unsaturated; a strong checker applied to a saturated model still nets negative. In open-ended reasoning domains, by contrast, the most effective architecture is independent sampling followed by mechanical aggregation, keeping intermediate artifacts out of the context of downstream models. Finally, many results that looked promising at first no longer reached statistical significance once three strict controls and a pre-registered procedure were applied together (16 of 32 runs were invalidated). That is an argument for experimental discipline when evaluating language-agent systems.

Limitations.

1. **Coverage of the deployable quantity Δ_{real} :** the experimental space of Δ_{real} has so far been surveyed on 15 directed model pairs on MBPP (0/15 of them reached a positive value that was statistically significant), one asymmetric pair on MATH (−12.4 points), and one same-capability configuration (+7.7 points). Three extended cross-checks did not finish because of the 14.6 GB VRAM limit of the free GPU: 8/9 generation branches are complete, and one branch remains — its result has not been read.
2. **Variability of the $\mathbf{A_gain}$ metric:** after standardising the `\boxed` output format, two independent runs recorded improvements of +0.8 and +8.0 points respectively. Both are non-negative, but the difference in magnitude (which comes from different data partitions and different model quantisation levels) means the claim that the aggregation role is neutral still needs confirmation at larger scale.
3. **How much variance in the loss L is explained:** the capability gap explains 82% of the variance in the improvement opportunity G , but only 35% of the variance in the exposure loss L . Most of the variation in L therefore still comes from architectural or protocol factors that we have not modelled fully.
4. **Statistical sensitivity of some control groups:** the role-permutation test on MATH has a relatively wide confidence interval (upper bound +13.4 points). Several in-depth control conditions have not been run in full, including an extra generation without the artifact, a context-length control under the exposure condition, and a comparison between different models within the same architecture family. In addition, the Shapley analysis (§5.3) currently rests on bootstrap over problem samples and has not been analysed over independent data folds.
5. **Cross-domain generalisation:** the analysis transferring the pattern to the mathematics domain rests on only 3 model pairs with wide confidence intervals; the positive region of the capability-gap regression currently has a single empirical point (+7.7 points at a gap of zero) and does not have the statistical sensitivity to be established firmly on MBPP; the sign-change threshold g^* has no parametric confidence interval attached.
6. **Decoding strategy and denominator stratification:** the information-flow branch of the study mainly uses a deterministic decoding strategy (greedy decoding), so each configuration has a single

point estimate; the main effects have not been stratified exhaustively over the subset of problems where an effect is possible.

7. **Range of model scales:** every empirical conclusion in this study was established on models from 1.5B to 32B parameters on structured reasoning tasks, so it cannot be extended directly to very large language models ($> 70\text{B}$); the training results (§5.11) in particular were all run on 0.5–1.5B models. The two evaluation standards we used (5-fold error bars and a pre-registration lock) also need more thorough cross-validation in follow-up work.

Member contributions

Dương Trọng Nguyên.

- Checking whether the roles actually perform the function they were assigned
- Designing and running the artifact-exposure experiment
- Calibrating the noise floor and controlling the denominator
- Report: abstract, measurement methods, experimental setup; terminology consistency throughout

Trương Đình Đức.

- Seven role-training methods and the shortcut analysis for each of them
- Directions tried and dropped: backward reasoning, solve-then-grade, problem decomposition, few-shot prompting for roles
- Writing the analysis notes for the experiments
- Report: introduction, related work, bibliography

Trần Tùng Dương.

- Experiment infrastructure on Kaggle and the results-collection system
- Scoring toolkit for the benchmarks
- Measuring exploitability, training the error classifier, routing experiments
- Report: synthesis, conclusion and limitations; presentation slides

Lê Hoàng Quân.

- Measuring role contributions with Shapley values and interaction indices
- Controlling the generation budget and the baselines
- The capability-gap pattern and the cross-domain check
- Report: results section; statistics toolkit

References

- [1] J. Austin et al. *Program Synthesis with Large Language Models*. arXiv:2108.07732, 2021.
- [2] Y. Benjamini, Y. Hochberg. *Controlling the False Discovery Rate*. JRSS–B 57(1), 1995.
- [3] M. Chen et al. *Evaluating Large Language Models Trained on Code*. arXiv:2107.03374, 2021.
- [4] Y. Chen et al. *When and Why Does Multi-Agent Debate Fail and Does It Really Underperform?* arXiv:2510.20963, 2025.
- [5] K. Cobbe et al. *Training Verifiers to Solve Math Word Problems*. arXiv:2110.14168, 2021.
- [6] Y. Du et al. *Improving Factuality and Reasoning in Language Models through Multiagent Debate*. ICML, 2024.
- [7] Z. Gou et al. *CRITIC: Large Language Models Can Self-Correct with Tool-Interactive Critiquing*. ICLR, 2024.
- [8] A. Grattafiori et al. *The Llama 3 Herd of Models*. arXiv:2407.21783, 2024.
- [9] D. Guo et al. *DeepSeek-Coder: When the Large Language Model Meets Programming*. arXiv:2401.14196, 2024.
- [10] D. Hendrycks et al. *Measuring Mathematical Problem Solving with the MATH Dataset*. NeurIPS, 2021.
- [11] J. Huang et al. *Large Language Models Cannot Self-Correct Reasoning Yet*. ICLR, 2024.
- [12] J. Kaplan et al. *Scaling Laws for Neural Language Models*. arXiv:2001.08361, 2020.
- [13] H. Lightman et al. *Let’s Verify Step by Step*. ICLR, 2024.

- [14] A. Madaan et al. *Self-Refine: Iterative Refinement with Self-Feedback*. NeurIPS, 2023.
- [15] *Multi-LLM-Agents Debate — Performance, Efficiency, and Scaling Challenges*. ICLR Blogposts, 2025.
- [16] *MASPRM: Multi-Agent System Process Reward Model*. arXiv:2510.24803, 2025.
- [17] *SHARP: Shapley Credit-based Optimization for Multi-Agent System*. arXiv:2602.08335, 2026.
- [18] L. S. Shapley. *A Value for n -Person Games*. Contributions to the Theory of Games II, 1953.
- [19] N. Shinn et al. *Reflexion: Language Agents with Verbal Reinforcement Learning*. NeurIPS, 2023.
- [20] C. Snell et al. *Scaling LLM Test-Time Compute Optimally Can Be More Effective than Scaling Model Parameters*. arXiv:2408.03314, 2024.
- [21] X. Wang et al. *Self-Consistency Improves Chain of Thought Reasoning in Language Models*. ICLR, 2023.
- [22] J. Wei et al. *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. NeurIPS, 2022.
- [23] A. Yang et al. *Qwen2.5 Technical Report*. arXiv:2412.15115, 2024.
- [24] L. Zheng et al. *Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena*. NeurIPS, 2023.

A Capability gap and cross-domain transfer

This appendix records the capability-gap regression and the cross-domain transfer test. Both were measured and are kept here for completeness; they support the main text rather than carrying a claim of their own. The transfer table (Table 19) is cited from the main text, and a reader following that pointer arrives here.

A pre-registered design over 6 models and 15 directed pairs, on the same 499 MBPP problems, regresses the opportunity term G on the capability gap and on a cross-family indicator:

$$\begin{aligned}
 G &= \beta_0 + \beta_1 \cdot \text{gap} + \beta_2 \cdot \text{cross_family} \\
 \beta_1 &= -0.1922 \ (p \approx 0) \\
 \beta_2 &= +0.0045 \ (\text{CI } [-0.0051; +0.0140]; \ p = 0.31).
 \end{aligned}$$

The gap term alone has $R^2 = 0.824$. The CI for β_2 lies entirely below the equivalence margin of $+0.02$ fixed in advance, which makes this an informative null: the cross-family effect is not separable from zero within that margin, and the explanatory variable is the capability gap. (The pool-diversity result — 3 models yield 2.70/3 distinct candidates versus 1.91/3 for a single model — is recorded only as a different-model effect; the different-model, same-family control has not been run.) The sign of β_1 is worth reading carefully — the larger the gap, the smaller the opportunity G , so the weaker the weak model, the fewer problems it can rescue for the strong one. Adding the repair branch for all 15 pairs (also pre-registered) gives the rule for the ceiling:

$$\Delta_{\text{ceil}} = +0.0218 - 0.2392 \times \text{gap}; \quad R^2 = 0.60; \ p = 10^{-5}; \ g^* \approx 0.09, \quad (2)$$

where $g^* = \beta_0/|\beta_1|$ is the gap at which the fitted line crosses zero (about 9 accuracy points). We have not estimated a CI for g^* , so it describes a trend and is not an operational rule. On MBPP, 0/15 pairs are positive at the 5% level and 3/15 are negative, so the rule can be stated only in its negative direction: once the gap exceeds ~ 0.09 , the repair protocol produces a negative effect. The gap explains 82% of the variance in G but only 35% of the variance in L ; most of the variation in the loss term comes from a factor we have not identified. (Mechanism note^(level B): the preservation threshold a repair pass has to clear grows with the gap twice as fast as its actual preservation ability — coefficients 2.18 versus 1.10; $p = 0.0066$ — which suggests the negative effect arises from the structure of the G – L budget rather than from any intrinsic degradation of the strong model.)

We then took the curve (2) fitted on MBPP and used it to predict Δ_{ceil} on maths problems without refitting (pre-registered; 3 Qwen pairs; paired bootstrap CI with 10 000 resamples). Table 19 gives the result.

Pair	Gap	Measured Δ_{ceil}	95% CI	MBPP prediction	Verdict
7B→14B	0.044	−0.0140	[−0.046; +0.018]	+0.0108	inside the interval
1.5B→7B	0.244	−0.1660	[−0.208; −0.124]	−0.0361	outside the interval
1.5B→14B	0.288	−0.0680	[−0.102; −0.034]	−0.0471	inside the interval

Table 19: A test of whether the capability-gap relationship carries over to the maths domain. All values are on a 0–1 scale. The prediction column comes from a curve fitted on MBPP and applied to these problems without refitting.

Two of the three predictions fall inside the 95% confidence interval, so the rule is not rejected. Two caveats stay attached to that statement. First, the intervals are wide (0.064–0.084), so the test has low resolution. Second, the three points are not monotonic: a gap of 0.244 produces a deeper negative effect than a gap of 0.288, so monotonicity of the rule is not confirmed on the maths domain. The 1.5B→7B pair falls outside in a systematic way. Its loss term has a point estimate of $L = 0.166$, and in the stratified table L covers 77 of 500 problems against G at 11 of 500 — a ratio of about 7. The same behaviour reappears in an independent measurement on the same pair ($-0.1380 \rightarrow -0.1660$). The relationship therefore describes a general trend only, and the exceptions appear when the weak model is far too weak for the difficulty of the problems.

B Code and data

All source code, summary results and experiment logs are published at <https://github.com/duongtrongnguyen123/multiagent-shapley-credit> (branch archive; the main branch contains only the report and the slides).